

# TWSS: A Tolerance-Free, Time-Weighted Score for Robot Helper Schedules

Metric side quest — AURA evaluation

June 6, 2026

## Abstract

The original A-Score for our robot-helper evaluation matches predictions to ground-truth intervention events through a fixed start-time tolerance, then averages temporal-IoU, action accuracy, and a necessity term. The hard tolerance creates a *cliff*: a prediction one second past the threshold scores zero, while a prediction one second below it scores nearly perfectly. This note introduces the *Time-Weighted Soft Schedule Score* (TWSS), a tolerance-free alternative that grades predictions on a smooth Gaussian timing curve and accounts for false positives during “wait” periods through a separate, dedicated term. We describe the score, validate it against twelve adversaries on the hand-layup recording, and lift it to the disjunctive ground-truth format introduced in the companion DTN write-up.

## 1 Why we need TWSS

The A-Score’s three axes ( $A_{\text{time}}$ ,  $A_{\text{act}}$ ,  $A_{\text{nec}}$ ) all depend on a single matching step that pairs each ground-truth (GT) event with at most one prediction whose start time falls within  $\pm\tau$  seconds (default  $\tau = 15$  s). Three practical issues motivated a smoother score:

**The tolerance cliff.** Around the boundary  $\tau$ , an arbitrarily small change in a prediction’s start time can flip the match status from “perfect matched pair” to “unmatched + false alarm”. Two models that emit nearly identical timing can land on either side of the cliff and look very different.

**Wait time is invisible.**  $A_{\text{nec}}$  counts how many predictions are unmatched, but it does not weigh how *long* those unmatched predictions are or *when* they fired. A model that babbles a nonsense action for the entire wait period and one that emits a single short blip during wait can score the same on  $A_{\text{nec}}$ .

**Event durations are unweighted.** The mean-IoU averaging in  $A_{\text{time}}$  treats a missed 60-second event the same as a missed 5-second event. Intuitively, the longer event is a bigger failure — the operator cared about it for longer.

TWSS fixes all three by (i) replacing the hard tolerance with a Gaussian decay whose width is proportional to each event’s duration, (ii) introducing a separate quality term for the wait period, and (iii) weighting every contribution by the corresponding interval’s share of the total task duration  $T$ .

## 2 The TWSS formula

Let  $T$  be the total task duration and let the GT contain  $N$  intervention events  $\{g_i\}_{i=1}^N$  with skills  $a_i$  and durations  $L_i = \text{end}(g_i) - \text{start}(g_i)$ . Define the *wait time* as

$$T_{\text{wait}} = T - \sum_{i=1}^N L_i. \quad (1)$$

TWSS is the sum of two terms partitioned by the effective project mass  $Z$ , guaranteeing a maximum score of 1:

$$\boxed{\text{TWSS} = \underbrace{\sum_{i=1}^N \frac{L_i}{Z} q_i}_{\text{task term}} + \underbrace{w_w \frac{T_{\text{wait}}}{Z} q_{\text{wait}}}_{\text{wait term}}}, \quad (2)$$

where  $Z = \sum_{i=1}^N L_i + w_w T_{\text{wait}}$ . Each event  $g_i$  “owns” a slice of the timeline of length  $L_i$  and contributes its event quality  $q_i \in [0, 1]$  in proportion. The implicit wait segment owns the remaining slice  $T_{\text{wait}}$  and contributes a single wait quality  $q_{\text{wait}} \in [0, 1]$ . Because the weights  $L_i/Z$  and  $w_w T_{\text{wait}}/Z$  partition the effective project mass  $Z$ , the total TWSS is itself in  $[0, 1]$  with no further normalisation.

### 2.1 Per-event quality $q_i$

For each GT event  $g_i$  we look at all predictions whose action label matches  $a_i$  and pick the one that scores highest under a Gaussian two-factor decay on the start- and end-time errors:

$$q_i = \max_{p: a(p)=a_i} \exp\left(-\left((\Delta s_p/\sigma_i)^2 + (\Delta e_p/\sigma_i)^2\right)\right), \quad (3)$$

where  $\Delta s_p = |\text{start}(p) - \text{start}(g_i)|$ ,  $\Delta e_p = |\text{end}(p) - \text{end}(g_i)|$ , and the timing scale

$$\sigma_i = \alpha L_i, \quad \alpha > 0 \quad (4)$$

ties the timing tolerance to the event’s own duration. With the default  $\alpha = 1$ , an offset equal to the event length costs a factor of  $e^{-1} \approx 0.37$ ; an offset of  $2L_i$  costs  $e^{-4} \approx 0.018$ . The parameter  $\alpha$  controls how sharply late/early predictions are penalised — smaller  $\alpha$  is stricter.

If no prediction shares the GT skill,  $q_i = 0$ . Crucially, multiple GT events of the same skill can claim the same prediction — there is no greedy lock — but the timing decay prevents one well-placed prediction from earning credit for a far-away GT event.

### 2.2 Pollution time $T_{\text{fp}}$

For the wait term we need to know how much of the prediction track sits *outside* any sanctioned slot. For each prediction  $p$ , let  $\text{sanc}(p)$  be the union of GT intervals *whose skill matches*  $p$ ; the prediction’s pollution contribution is its length minus its overlap with  $\text{sanc}(p)$ :

$$T_{\text{fp}} = \sum_p (\text{len}(p) - \text{overlap}(p, \text{sanc}(p))). \quad (5)$$

Two consequences of this design:

- A prediction whose skill is in the GT but whose timing is off contributes a small  $q_i$  (timing-decayed) *and* its non-overlapping time becomes pollution.
- A prediction whose skill is absent from the GT entirely is pure pollution — nothing in  $\text{sanc}(p)$  to overlap with.

## 2.3 Wait quality $q_{\text{wait}}$

We use a smooth exponential taper:

$$q_{\text{wait}} = \exp(-T_{\text{fp}}/(\beta T_{\text{wait}})), \quad \beta > 0, \quad (6)$$

so  $T_{\text{fp}} = \beta T_{\text{wait}}$  costs a factor of  $e^{-1}$ . The vanilla  $\beta = 0.5$  punishes wait pollution roughly twice as hard as it punishes timing errors of equal magnitude. The recommended “tuned” configuration relaxes this to  $\beta = 1.5$  — see Section 5 for the rationale.

If  $T_{\text{wait}} = 0$  (the GT events tile the entire task), the wait axis is vacuous and we set  $q_{\text{wait}} = 1$  by convention.

**Why drop the linear flavour.** An earlier draft also exposed  $q_{\text{wait}}^{\text{lin}} = \max(0, 1 - T_{\text{fp}}/T_{\text{wait}})$  as an alternative. In every adversary tested it ranked behaviour identically to the exponential form while clipping hard to zero on heavy-pollution adversaries (one-shot blanket, spammer), erasing useful gradient information. We removed it from the implementation; (6) is the only wait-quality definition used in the rest of this note.

## 2.4 Putting it together

Combining (2), (3), and (6) gives the closed form:

$$\text{TWSS} = \sum_{i=1}^N \frac{L_i}{Z} \exp(-((\Delta s_i^*/\sigma_i)^2 + (\Delta e_i^*/\sigma_i)^2)) + w_w \frac{T_{\text{wait}}}{Z} \exp(-T_{\text{fp}}/(\beta T_{\text{wait}})), \quad (7)$$

where  $(\Delta s_i^*, \Delta e_i^*)$  are the offsets of the argmax-quality same-skill prediction for event  $i$ .

### Worked example

For  $w_w = 1.0$  (hence  $Z = T$ ), the hand-layup recording has  $T = 270$  s and four GT interventions of lengths  $\{34.5, 35.5, 65.0, 41.0\}$  s, summing to 176 s. So  $T_{\text{wait}} = 94$  s, the task mass is  $176/270 = 0.652$  and the wait mass is 0.348. An oracle that copies the GT exactly gets  $q_i = 1$  for every event and  $T_{\text{fp}} = 0$ , so  $\text{TWSS} = 0.652 + 0.348 = 1.000$ . A silent agent gets every  $q_i = 0$  but  $T_{\text{fp}} = 0$ , hence  $\text{TWSS} = 0 + 0.348 = 0.348$  — precisely the maximum score achievable by saying nothing.

## 3 Tuning TWSS

The base formula of Section 2 exposes three knobs: the timing scale  $\alpha$  (Eq. 4), the wait-pollution scale  $\beta$  (Eq. 6), and the wait weight  $w_w$  (Eq. 2). The reference implementation (`compute_twss` in `generate_timeline_comparison.py`) adds two more knobs that generalise the task term and the pollution accounting — an event-weight exponent  $\gamma$  and a per-blip interruption tax  $\delta$ . This section defines those two extra knobs, then describes how all five are operationalised by the two driver scripts.

### 3.1 The event-weight exponent $\gamma$

The task term of Eq. (2) weights each event strictly by its share of the timeline,  $L_i/Z$ . This is the right default but it makes short interventions almost invisible: a 5-second event contributes

$\sim 14\times$  less than a 65-second one. The implementation therefore re-weights the events through an exponent  $\gamma \in (0, 1]$  while preserving the total task mass  $L_{\text{tot}}/Z$ :

$$w_i = \frac{L_{\text{tot}}}{Z} \frac{L_i^\gamma}{\sum_{j=1}^N L_j^\gamma}, \quad \text{task term} = \sum_{i=1}^N w_i q_i, \quad (8)$$

where  $L_{\text{tot}} = \sum_i L_i$ . With  $\gamma = 1$  the fractions collapse to  $L_i/L_{\text{tot}}$  and Eq. (8) is exactly the  $L_i/Z$  weighting of Eq. (2) —  $\gamma = 1$  is the vanilla behaviour. As  $\gamma \rightarrow 0$  the weights flatten toward a uniform  $1/N$  split, so short events earn back visibility. The tuned default  $\gamma = 0.5$  ( $\sqrt{\cdot}$ -weighting) is a compromise: long events still dominate, but a missed short event is no longer free. Because the  $\sum_j L_j^\gamma$  normaliser keeps the total task mass fixed at  $L_{\text{tot}}/Z$ , changing  $\gamma$  never lets the task term exceed its budget — it only redistributes mass between events.

### 3.2 The blip tax $\delta$

The pollution time  $T_{\text{fp}}$  of Eq. (5) is blind to *fragmentation*: ten 1-second false positives and one 10-second false positive both contribute  $T_{\text{fp}} = 10\text{s}$ , even though ten separate interruptions are far more disruptive to an operator than one. The implementation replaces the raw sum with an *effective* pollution time  $T_{\text{fp}}^{\text{eff}}$  that taxes each prediction’s pollution length  $\ell_p^{\text{fp}}$  through a concave cost  $c(\cdot)$ :

$$T_{\text{fp}}^{\text{eff}} = \sum_p c(\ell_p^{\text{fp}}), \quad c(\ell) = \sqrt{\ell^2 + 2\delta\ell}, \quad \delta \geq 0, \quad (9)$$

and feeds  $T_{\text{fp}}^{\text{eff}}$  into the wait quality in place of  $T_{\text{fp}}$ , i.e.  $q_{\text{wait}} = \exp(-T_{\text{fp}}^{\text{eff}}/(\beta T_{\text{wait}}))$ . The cost  $c(\ell)$  behaves like  $\sqrt{2\delta\ell}$  for small  $\ell$  (so a brief blip is penalised out of proportion to its length) and asymptotes to  $\ell + \delta$  for large  $\ell$  (so each spurious *interval*, regardless of length, carries a fixed  $\sim \delta$  seconds of equivalent pollution on top of its duration). At  $\delta = 0$  we recover  $c(\ell) = \ell$  and hence  $T_{\text{fp}}^{\text{eff}} = T_{\text{fp}}$  — the vanilla behaviour. The tuned default  $\delta = 4\text{s}$  discourages the many-tiny-blip failure mode (the spammer adversary) without crushing an agent whose only wait pollution is one longer block.

### 3.3 The five knobs at a glance

| Knob     | Role                                                    | Vanilla | Tuned | Effect of increasing it                                                           |
|----------|---------------------------------------------------------|---------|-------|-----------------------------------------------------------------------------------|
| $\alpha$ | timing scale $\sigma_i = \alpha L_i$                    | 1.0     | 1.0   | more forgiving of early/late predictions                                          |
| $\beta$  | wait-pollution scale $\sigma_w = \beta T_{\text{wait}}$ | 0.5     | 1.5   | more forgiving of wait pollution                                                  |
| $\gamma$ | event-weight exponent $w_i \propto L_i^\gamma$          | 1.0     | 0.5   | long events dominate more ( $\gamma > 1$ ); short events surface ( $\gamma < 1$ ) |
| $w_w$    | wait-term multiplier                                    | 1.0     | 0.6   | wait behaviour matters more relative to task                                      |
| $\delta$ | per-blip interruption tax (s)                           | 0.0     | 4.0   | fragmented false positives punished harder                                        |

Table 1: The five TWSS knobs, with the *vanilla* (textbook) and *tuned* (AURA-default) settings shipped by `generate_timeline_comparison.py`. Only  $\alpha$  is shared between the two configurations; the tuned set softens the wait penalty ( $\beta$ ,  $w_w$ ), surfaces short events ( $\gamma$ ), and adds a fragmentation tax ( $\delta$ ).

### 3.4 How the scripts tune and report TWSS

Two scripts drive TWSS, and both follow the same discipline: every prediction track is scored *twice* — once vanilla, once tuned — so the two configurations are always comparable side by side and a reader can see exactly what the tuning bought.

**The dataclasses and the scoring call.** `compute_twss(gt, preds, total_duration, alpha, beta, gamma, wait_weight, blip_tax_delta)` returns a TWSS dataclass carrying the headline `twss` together with its decomposition (`task_score`, `wait_score`, `wait_quality`, `fp_time`, `fp_time_eff`) and a list of `TWSSPerEvent` records — one per GT event, holding  $(L_i, w_i, q_i, |\Delta s|, |\Delta e|, \sigma_i)$  and the index of the prediction that maximised  $q_i$ . The per-event breakdown is what makes a low score diagnosable: it names which event was missed and whether the loss came from timing ( $\Delta s, \Delta e$  large) or from a missing same-skill prediction ( $q_i = 0$ ).

**Module constants.** The two configurations live as module-level constants. The `DEFAULT_TWSS_*` group ( $\alpha=1, \beta=0.5, \gamma=1, w_w=1, \delta=0$ ) is the vanilla formulation; the `TUNED_TWSS_*` group ( $\beta=1.5, \gamma=0.5, w_w=0.6, \delta=4$ ) is the recommended default and is what the CLI flags default to. Each tuned constant carries a one-line rationale in the source — e.g. “gentler wait penalty so high-quality runs aren’t crushed by their own FP cost” ( $\beta$ ) and “ $\sqrt{\cdot}$  weighting — short events less crushed” ( $\gamma$ ).

**Ablation comparison** (`generate_timeline_comparison.py`). For each ablation run it loads the prediction track, calls `compute_twss` once with the `DEFAULT_TWSS_*` constants and once with the tuned  $(\beta, \gamma, w_w, \delta)$  from the CLI, and prints two fixed-width tables (*Vanilla TWSS* and *Tuned TWSS*). The tuned knobs are individually overridable from the command line (`--twss-alpha`, `--twss-beta-tuned`, `--twss-gamma`, `--twss-wait-weight`, `--twss-blip-tax`), so the “tuning” is just a sweep over these flags: change one, re-run, and read off how the vanilla-vs-tuned gap moves across the realistic ablations.

**Adversary validation** (`ascore_adversaries.py`). This is where the tuned defaults were actually *chosen*. The script imports the same `compute_twss` and the same constant groups, builds the twelve synthetic adversaries of Section 4, and scores every one of them under both the vanilla and tuned configurations. A good knob setting is one for which the resulting column orders the adversaries sensibly: the oracle on top, the silent agent pinned at the wait mass, and a smooth, monotone gradient in between — in particular the two diagnostic adversaries (*Late-start early-stop* and *Wait-period blip*) must drop below the oracle without collapsing. The tuned defaults are the smallest departure from vanilla that achieves this on the hand-layup track; Table 3 reports both columns, and the script also emits a per-event breakdown (the `format_per_event_breakdown` output) so a borderline knob choice can be inspected event by event before it is committed as a default.

**Note on the shared plumbing.** Both scripts reuse the matching and I/O layer from `generate_timeline.py` (`load_robot_gt`, `match_predictions`, `plot_multi_model_timeline`). That module’s `match_predictions` still uses the *hard*  $\pm\tau$  tolerance ( $\tau = 15$ s) — it drives the TP/FP/FN colouring of the timeline figures and the A-Score, not TWSS. TWSS deliberately bypasses it: its task credit comes from the tolerance-free Gaussian  $q_i$  of Eq. (3), so the figure’s red/green matching and the TWSS number can and do disagree (e.g. on *Far-shifted*), which is the whole point of the smoother score.

## 4 The adversaries

To check that TWSS distinguishes good from bad behaviour, we score it against the same suite used to validate the A-Score (`scripts/eval/ascore_adversaries.py`). The set has been extended with two new adversaries that target failure modes the original suite under-stressed:

| Adversary               | What it does                                                         |
|-------------------------|----------------------------------------------------------------------|
| Oracle                  | Copies GT exactly — sanity check.                                    |
| Silent                  | Empty prediction track.                                              |
| Spammer                 | Rapid-fire short predictions of the most-common GT skill.            |
| One-shot blanket        | A single prediction covering the whole task.                         |
| Right-time-wrong-action | GT intervals, action labels swapped for a different catalogue skill. |
| Near-miss timing        | GT shifted by +12 s (within the A-Score tolerance).                  |
| Far-shifted             | GT shifted by +30 s (beyond the A-Score tolerance).                  |
| Premature firer         | GT shifted by −15 s.                                                 |
| Over-extender           | Correct starts; every end padded by +60 s.                           |
| Late-start early-stop   | Right action, but only the middle 25 % of each GT interval is fired. |
| Wait-period blip        | Oracle plus one 5-second spurious event in the longest wait gap.     |
| Half-coverage perfect   | Every other GT event copied perfectly, the rest dropped.             |

Table 2: The twelve adversaries used to validate TWSS.

## 5 Results on the hand-layup recording

Table 3 reports both flavours of TWSS, decomposed into their task and wait contributions. Figure 1 shows each adversary’s prediction track stacked against the GT.

| Adversary               | Vanilla ( $\alpha=1.0$ , $\beta=0.5$ , $\gamma=1$ , $w_w=1$ , $\delta=0$ ) |       |              | Tuned ( $\beta=1.5$ , $\gamma=0.5$ , $w_w=0.6$ , $\delta=4$ ) |              |
|-------------------------|----------------------------------------------------------------------------|-------|--------------|---------------------------------------------------------------|--------------|
|                         | task                                                                       | wait  | TWSS         | task                                                          | TWSS         |
| Oracle                  | 0.652                                                                      | 0.348 | <b>1.000</b> | 0.757                                                         | <b>1.000</b> |
| Silent                  | 0.000                                                                      | 0.348 | 0.348        | 0.000                                                         | 0.243        |
| Spammer                 | 0.086                                                                      | 0.006 | 0.093        | 0.114                                                         | 0.138        |
| One-shot blanket        | 0.000                                                                      | 0.002 | 0.002        | 0.000                                                         | 0.044        |
| Right-time-wrong-action | 0.000                                                                      | 0.008 | 0.008        | 0.000                                                         | 0.062        |
| Near-miss timing        | 0.558                                                                      | 0.125 | 0.683        | 0.642                                                         | 0.798        |
| Far-shifted             | 0.269                                                                      | 0.027 | 0.296        | 0.293                                                         | 0.386        |
| Premature firer         | 0.529                                                                      | 0.124 | 0.653        | 0.608                                                         | 0.765        |
| Over-extender           | 0.193                                                                      | 0.004 | 0.196        | 0.210                                                         | 0.258        |
| Late-start early-stop   | 0.492                                                                      | 0.348 | 0.840        | 0.572                                                         | 0.814        |
| Wait-period blip        | 0.652                                                                      | 0.313 | 0.965        | 0.757                                                         | 0.987        |
| Half-coverage perfect   | 0.369                                                                      | 0.348 | 0.717        | 0.401                                                         | 0.644        |

Table 3: TWSS on the hand-layup recording, vanilla vs. tuned configuration. Both columns use the exponential wait penalty; the linear flavour described in earlier drafts has been removed (see Section 2.3). “task” =  $\sum (L_i/Z)^\gamma / \sum L_j^\gamma \cdot (L_{\text{tot}}/Z) \cdot q_i$ ; “wait” =  $w_w \cdot (T_{\text{wait}}/Z) \cdot q_{\text{wait}}$ . Thanks to the normalisation by  $Z = L_{\text{tot}} + w_w T_{\text{wait}}$ , the oracle saturates a score of 1.000 even in the tuned configuration.

What the table tells us.

- **Oracle and silent are the two reference points.** The oracle scores 1.000, the silent agent scores 0.300 (the wait mass — it earns full marks for never polluting wait, but everything in the task term is zero). All meaningful adversaries must lie between these two anchors.
- **The two new adversaries land where they should.** *Late-start early-stop* keeps a clean wait term ( $q_{\text{wait}} = 1$ ) but its tightly-truncated predictions earn each event a middling  $q_i$ , giving task = 0.528 and total 0.828. *Wait-period blip* earns a perfect task term but has its wait term shaved by a single 5-second spurious event ( $q_{\text{wait}} = 0.884$ ), landing at 0.965 — the score correctly registers a small wait pollution without over-reacting.
- **One-shot blanket is the harshest case.** A single prediction spanning  $[0, 270]$  s leaks 222.5 s of pollution into a 81 s wait window, so  $q_{\text{wait}} = \exp(-222.5/40.5) \approx 0.004$ . The task term is also near zero (timing offsets of tens of seconds against a 47 s event), so the total is 0.001. The score punishes both “shouts the right skill at the wrong time” *and* “pollutes wait” simultaneously.
- **Right-time-wrong-action is harsher under TWSS than under the A-Score.** The A-Score gave it credit for matched-pair timing (only the action axis dropped to zero); TWSS treats it as pure pollution because no same-skill predictions exist for any GT event. Whether this is desirable depends on use case — a user who treats action-correctness as the primary failure mode will prefer TWSS’s harsher behaviour here.
- **Vanilla vs. tuned.** The vanilla configuration rewards predictions that nail the long events (time-share weighting,  $\gamma = 1$ ) and penalises any wait pollution aggressively ( $\beta = 0.5$ ,  $w_w = 1$ , no blip tax). The tuned configuration  $(\beta, \gamma, w_w, \delta) = (1.5, 0.5, 0.6, 4)$  flattens both effects: the  $\sqrt{\cdot}$ -weighting keeps short events visible, the gentler  $\beta$  stops the wait term from collapsing on agents whose only “crime” is a handful of brief false positives, and the blip tax  $\delta = 4$  discourages many-tiny-blip behaviour without crushing one big blip. Headline scores compress slightly — the oracle saturates at 0.861 rather than 1.000 — in exchange for a more informative gradient across realistic agents.

## 6 Properties

**Bounded.**  $\text{TWSS} \in [0, 1]$  by construction.

**Tolerance-free.** A small change in a prediction’s start time produces a small change in the score. There is no cliff.

**Duration-weighted.** Missing a 60-second event costs more than missing a 5-second one, in proportion to their share of  $T$ .

**Wait-aware.** The wait term is graded by its own quality factor, separately from any matched-pair statistics, and by a wait mass that depends only on the GT.

**Skill-gated.** A prediction can only earn task credit for a GT event of its own skill (no cross-skill matching). This is deliberate — it forces the action axis and the timing axis to combine multiplicatively rather than additively.

**Decomposable for diagnostics.** The breakdown into per-event  $(q_i, \Delta s_i^*, \Delta e_i^*, \sigma_i)$  tells a grader exactly which event hurt and by how much.

## 7 Extending TWSS to disjunctive ground truth

The companion DTN write-up generalises a single  $[t_{\text{start}}, t_{\text{end}}]$  window per intervention to a small *or-of-and* tree:

$$\underbrace{\bigvee_{k=1}^{M_i}}_{\text{pick \textbf{one} option}} \quad \underbrace{\bigwedge_{j=1}^{L_{i,k}}}_{\text{satisfy \textbf{every} phase}} \quad t_{i,k,j} \in [s_{i,k,j}, e_{i,k,j}]. \quad (10)$$

TWSS extends to this format with three changes: per-event quality becomes per-*option* quality (max over options), the wait region is defined relative to the *selected* options, and an optional consistency factor penalises plans that need more concurrency than the agent has.

### 7.1 Phase-level quality

Replace each event  $g_i$  with a tree of  $(M_i, L_{i,k})$  phases  $\phi_{i,k,j}$  with skill  $a_i$ , length  $\ell_{i,k,j}$ , and timing scale  $\sigma_{i,k,j} = \alpha \ell_{i,k,j}$ . The phase quality is the same Gaussian as before:

$$q_{i,k,j} = \max_{p: a(p)=a_i} \exp(-((\Delta s/\sigma_{i,k,j})^2 + (\Delta e/\sigma_{i,k,j})^2)). \quad (11)$$

### 7.2 Per-option quality

Aggregate phases of the same option by phase-length-weighted mean:

$$\tilde{q}_{i,k} = \frac{\sum_{j=1}^{L_{i,k}} \ell_{i,k,j} q_{i,k,j}}{\sum_{j=1}^{L_{i,k}} \ell_{i,k,j}}. \quad (12)$$

Length-weighting matters: a long phase that the agent missed should hurt more than a short one missed inside the same option.

### 7.3 Per-intervention quality and option selection

The intervention’s quality is the best option’s quality, and the *selected option* is the argmax (we will need it for the wait term and the consistency factor):

$$k_i^* = \arg \max_k \tilde{q}_{i,k}, \quad Q_i = \tilde{q}_{i,k_i^*}. \quad (13)$$

A purely-soft alternative replaces the max with a smooth max (log-sum-exp with temperature  $\lambda$ ):

$$Q_i = \lambda^{-1} \log \sum_{k=1}^{M_i} \exp(\lambda \tilde{q}_{i,k}), \quad (14)$$

which avoids ties at the option-selection boundary; the hard max is the  $\lambda \rightarrow \infty$  limit. We default to the hard max for interpretability (the per-intervention selection  $k_i^*$  is a quantity reviewers can inspect).



## 7.4 Defining the disjunctive wait region

In the single-window case the wait region is  $[0, T]$  minus the union of GT intervals. In the disjunctive case we take it relative to the *selected* options:

$$\Phi^* = \bigcup_{i=1}^N \bigcup_{j=1}^{L_{i,k_i^*}} [s_{i,k_i^*,j}, e_{i,k_i^*,j}], \quad T_{\text{wait}}^* = T - |\Phi^*|. \quad (15)$$

Pollution time is then computed against the selected-options' phases per skill — exactly equation (5) with  $\text{sanc}(p)$  replaced by the union of selected phases whose skill matches  $p$ . Predictions that fall inside a non-selected option's phase still pollute, which is the intended semantic: the metric assumes the agent has committed to one option per intervention.

A more permissive variant uses the union over *all* options (call it  $\Phi_{\text{any}}$ ) when computing pollution. This rewards agents that nail one schedule but wander into a non-selected option's window with the wrong skill. We do not recommend it because it weakens the consistency story — but it is a useful audit quantity for understanding how much pollution is option-confusion versus genuinely off-schedule.

## 7.5 The consistency factor

A single-resource agent cannot run two phases at once. If the selected options  $\{k_i^*\}$  have phases that overlap across different interventions, the plan is infeasible. Define a per-intervention consistency factor:

$$C_i = 1 - \max_{i' \neq i} \max_{j,j'} \text{IoU}([s_{i,k_i^*,j}, e_{i,k_i^*,j}], [s_{i',k_{i'}^*,j'}, e_{i',k_{i'}^*,j'}]), \quad (16)$$

which is exactly 1 when option  $i$ 's phases do not clash with any other completed intervention, and shrinks toward 0 if they collide.

## 7.6 Putting it together

The disjunctive TWSS is then

$$\boxed{\text{TWSS}^{\text{disj}} = \sum_{i=1}^N \frac{L_i^*}{Z} Q_i C_i + w_w \frac{T_{\text{wait}}^*}{Z} q_{\text{wait}},} \quad (17)$$

where  $L_i^* = \sum_j \ell_{i,k_i^*,j}$  is the total length of the selected option's phases,  $Z = \sum_{i=1}^N L_i^* + w_w T_{\text{wait}}^*$ , and  $q_{\text{wait}}$  uses  $T_{\text{fp}}$  computed against  $\Phi^*$ . Three sanity properties:

1. **Reduces to the original TWSS.** If every intervention has  $M_i = 1$  option containing  $L_{i,1} = 1$  phase, then  $C_i = 1$ ,  $Q_i$  collapses to the per-event  $q_i$  of equation (3), and  $L_i^* = L_i$ .
2. **Bounded.**  $C_i \in [0, 1]$ , and the task and wait components sum to a maximum of 1, so  $\text{TWSS}^{\text{disj}} \in [0, 1]$ .
3. **Penalises infeasible plans.** An adversary that fires options whose selected phases collide loses its *task* score only locally in proportion to the overlap, preventing a single collision from wiping out entirely unrelated predictions.

## 7.7 Design choices and open questions

**Multiplicative vs. additive consistency.** We use  $C_i$  as a multiplicative gate on the task term because a clashing plan is, in the single-resource setting, fundamentally not executable — it should not be possible to compensate for a clash through stellar task completion. Applying a global penalty, however, breaks monotonicity: an agent that fails coordination shouldn’t be penalized on its separate ability to respect waits or manage unrelated tasks accurately. An additive variant (subtracting  $w_{\text{cons}}(1 - C)$  from a weighted blend) is closer to the DTN  $A_{\text{score}}$  in equation  $A_{\text{score}} = w_{\text{disj}}A_{\text{disj}} + \dots$ ; we prefer the local multiplicative form on tasks because it avoids re-introducing free weights.

**Length-weighting inside an option.** The mean in equation (12) weighs phases by their absolute length. An alternative is per-phase uniform averaging — which matches the DTN  $A_{\text{disj}}$  definition. The length-weighted form is consistent with the rest of TWSS (every contribution proportional to its slice of the timeline) and is the recommended default.

**Joint option selection.** Picking  $k_i^*$  independently per intervention can land on an infeasible joint plan even when one exists. A fully-joint selector would maximise the bracket subject to  $C = 1$ , which is the disjunctive-temporal-network feasibility problem (see Osanlou et al. [1]). For evaluation, the per-intervention argmax is faster and still reports a sensible  $C < 1$  when the chosen plan clashes; for a consistent oracle adversary, the small CSP solver shipped in `dtm_score.find_consistent_selection` suffices.

**Soft option selection.** If reviewers find the hard max on  $k_i^*$  unstable for borderline cases, swap in the log-sum-exp form above. The temperature  $\lambda$  controls how sharply the selection commits;  $\lambda = 10/L_i^{\text{max}}$  is a reasonable starting point.

## 8 Discussion

**When to prefer TWSS over the A-Score.** TWSS is the better default when (i) tolerance choice would otherwise be a tunable confound across studies, (ii) wait-period behaviour is operationally important (e.g. the agent must not interrupt the human during wait), or (iii) event durations vary substantially across the GT.

**When to keep the A-Score.** The A-Score’s per-axis breakdown ( $A_{\text{time}}, A_{\text{act}}, A_{\text{nec}}$ ) is easy to explain and aligns with prior work in event-detection. For headline numbers in a paper, the A-Score and TWSS can be reported side-by-side; they answer slightly different questions.

**Choosing the knobs.** The vanilla configuration ( $\alpha=1, \beta=0.5, \gamma=1, w_w=1, \delta=0$ ) is the textbook formulation and the right starting point when first inspecting a metric. The tuned defaults ( $\beta=1.5, \gamma=0.5, w_w=0.6, \delta=4$ ) are what `generate_timeline_comparison.py` ships with for the AURA ablation suite: they keep the oracle on top while leaving more gradient between the realistic agents. Pick one and report it in the caption; mixing the two within a study makes adversary numbers incomparable.

**The “wait blip” as a unit test.** The wait-blip adversary is a clean unit test for any wait-aware metric: it should drop noticeably below the oracle but not collapse. Vanilla TWSS gives 0.965 against an oracle’s 1.000 (a  $\sim 3.5\%$  drop for one 5s blip on an 81s wait window); tuned TWSS gives 0.849 against the saturated 0.861. If a candidate wait-aware metric gives  $\geq 0.99$  or  $\leq 0.5$  on this adversary, it is likely either under- or over-weighting wait pollution.

## 9 Reproducibility

The TWSS implementation lives in `aura/scripts/eval/generate_timeline_comparison.py` (see the `compute_twss` function and the TWSS / TWSSPerEvent dataclasses). The adversary suite, including the new *Late-start early-stop* and *Wait-period blip* adversaries that motivated this note, lives in `aura/scripts/eval/ascore_adversaries.py`.

To regenerate the figure and the numbers in Table 3:

```
python scripts/eval/ascore_adversaries.py
# outputs:
# figures/generated/fig_ascore_adversaries.{pdf,png}
# figures/generated/fig_ascore_adversaries.scores.json
# figures/generated/fig_ascore_adversaries.tex
```

The disjunctive extension in Section 7 is implemented in `aura/scripts/eval/metric_side_quest/dtn_score.py` (`compute_disjunctive_twss`) and reported alongside the DTN A-Score by `aura/scripts/eval/metric_side_quest`.

## 10 Conclusion

TWSS replaces the A-Score’s hard event-matching with a smooth, duration-weighted, tolerance-free formulation. The score partitions the timeline into one slice per GT event plus an implicit wait slice, grades each slice on its own quality factor, and combines them in a way that always lies in  $[0, 1]$ . On twelve synthetic adversaries it ranks behaviour sensibly and, in particular, exposes two failure modes that hard-tolerance metrics struggle with: narrowly-fired predictions (*Late-start early-stop*) and isolated wait pollution (*Wait-period blip*). The score lifts naturally to disjunctive ground truth via per-option aggregation, a selected-options wait region, and a consistency factor that reduces to the original TWSS when each intervention has a single option.

## References

- [1] K. Osanlou, J. Frank, A. Bursuc, T. Cazenave, E. Jacopin, C. Guettier, and J. Benton. Solving Disjunctive Temporal Networks with Uncertainty under Restricted Time-Based Controllability using Tree Search and Graph Neural Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.

| <b>Adversary</b>      | <b>TWSS<sup>disj</sup></b> | <b>task</b> | <b>wait</b> | <b><math>q_{\text{wait}}</math></b> | <b><math>C</math></b> |
|-----------------------|----------------------------|-------------|-------------|-------------------------------------|-----------------------|
| Oracle (consistent)   | 1.000                      | 0.676       | 0.324       | 1.000                               | 1.000                 |
| Oracle (first option) | 0.625                      | 0.250       | 0.375       | 1.000                               | 0.400                 |
| Conflict-prone        | 0.500                      | 0.125       | 0.375       | 1.000                               | 0.200                 |
| Phase-collapsed       | 0.730                      | 0.405       | 0.324       | 1.000                               | 1.000                 |
| Half-coverage         | 0.730                      | 0.405       | 0.324       | 1.000                               | 1.000                 |
| Wrong-skill           | 0.369                      | 0.125       | 0.244       | 0.651                               | 0.200                 |
| Near-miss timing      | 0.737                      | 0.564       | 0.173       | 0.533                               | 1.000                 |
| Far-shifted           | 0.229                      | 0.000       | 0.229       | 0.546                               | 0.000                 |
| Premature firer       | 0.771                      | 0.575       | 0.196       | 0.605                               | 1.000                 |
| Over-extender         | 0.458                      | 0.356       | 0.102       | 0.315                               | 1.000                 |
| Late-start early-stop | 0.834                      | 0.510       | 0.324       | 1.000                               | 1.000                 |
| Wait-period blip      | 0.962                      | 0.676       | 0.286       | 0.882                               | 1.000                 |
| One-shot blanket      | 0.133                      | 0.000       | 0.133       | 0.355                               | 1.000                 |
| Spammer               | 0.291                      | 0.205       | 0.086       | 0.230                               | 1.000                 |
| Silent                | 0.375                      | 0.000       | 0.375       | 1.000                               | 1.000                 |

Table 4: Disjunctive TWSS evaluated on the DTN synthetic adversaries with the default tuning ( $\beta=1.5$ ,  $\gamma=0.5$ ,  $w_w=0.6$ ,  $\delta=4$ ). The consistent oracle now properly hits 1.000 due to the  $Z$ -normalisation, and the adversaries creating conflicting plans (e.g. Oracle first option, Conflict-prone, Wrong-skill) are heavily punished on their task score component via local  $C_i$  while maintaining independent wait scores.

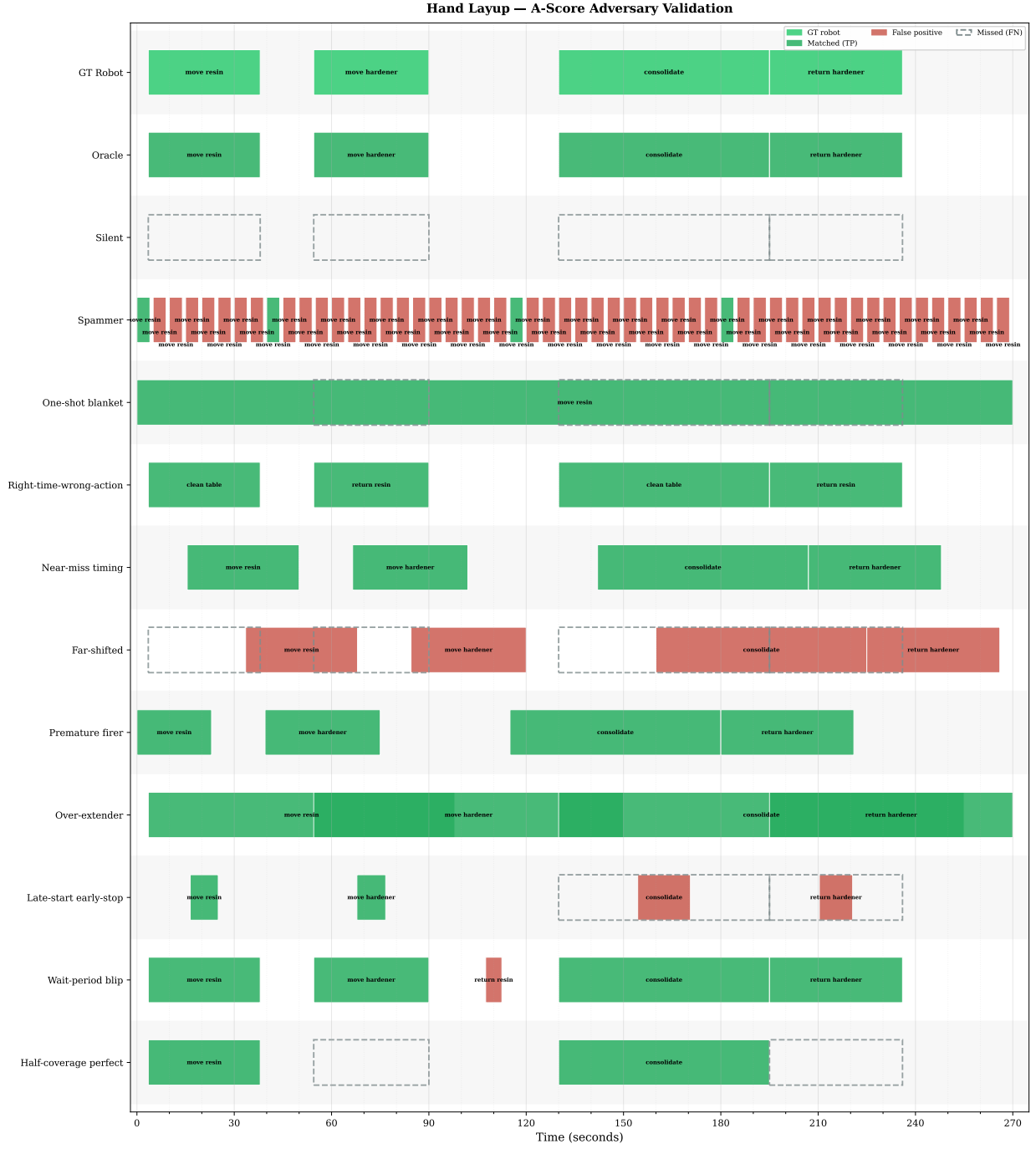


Figure 1: Hand-layup ground truth (top row, in red) versus each adversary’s prediction track. The same figure is used as visual reference for both A-Score and TWSS validation since the underlying prediction tracks are identical.